DeCal: Decoupled Tail Budget Control for Non-Stationary LLM Serving

Anonymous Author(s)

Motivation. Interactive LLM serving is often limited by tail sequence lengths rather than by mean request volume. A small number of long generations can dominate KV-cache demand, yet the capacity envelope for the next window must be chosen before that window is observed. Under-budgeting exposes the engine to queueing and KV-cache pressure. Over-budgeting ties up scarce KV capacity and reduces concurrency and utilization to insure against tails that may never materialize. We study this decision as a completed-history control problem in which each budget entity receives one scalar tail-length proxy budget from completed history. The budget is not a request scheduler or KV allocator. Instead, it is an upstream capacity signal consumed by mechanisms that operate after demand becomes visible.

Across two public LLM-serving workload sources, next-window tail demand changes frequently enough to require continuous control rather than one-time calibration [9, 10]. ServeGen's language m-mid subset contains 92 clients and 2,840 pairs of adjacent windows after preprocessing. Defining drift by adjacent-window Jensen–Shannon divergence JS ≥ 0.1, 42.2% of its transitions are drift events, and 17.7% remain above the stricter JS ≥ 0.3 threshold. Its clients also exhibit partial rather than global similarity. Nearby clients can provide bounded startup or recovery priors, whereas one pooled budget would obscure client-specific tails. BurstGPT provides a complementary long-horizon view with 11,925 adjacent-window pairs from four production-style streams. Its corresponding drift rates are 89.4% and 82.1%, showing sustained regime movement across the observed streams. Together, these traces motivate temporary protection for newly exposed tail regimes, retraction when risk subsides, and entity-local learning with only selective transfer.

Limitations of current approaches. Existing work does not pin down the calibration reference created by a protected budget. Execution-layer systems improve capacity use after requests are visible [6–8, 14]. Forecasting improves the base estimate [2, 11, 12], while online calibration corrects historical errors [1, 3–5, 13]. The missing piece is the reference used by that correction after the controller deliberately raises the current budget for protection. The realized tail $q_{0.99}(P_{c,t})$ enters history only after the budget for window t has already been emitted. This timing creates feedback contamination. A coupled controller raises the current budget when drift is detected and then scores the realized tail against that raised budget. If protection succeeds, the recorded error becomes artificially small, so later corrections contract exactly when the tail regime has changed. Better drift detection does not remove this problem. The controller must protect the current window while keeping future learning tied to an unprotected reference.

Key idea. We propose DeCal, a drift-aware tail budget controller built around a simple invariant. Protection is an action rather than a learning target. Before window t starts, DeCal emits

$$B_{c,t} = B^{\text{base}}_{c,t}\gamma_{c,t} + C_{c,t}, \tag{1}$$

where $B^{\text{base}}_{c,t}$ is the recent-tail estimate. The multiplier $\gamma_{c,t} \geq 1$ provides temporary drift protection, and $C_{c,t}$ is the calibration offset. Only completed-history evidence can raise $\gamma_{c,t}$, and the raise is explicitly retractable. Calibration uses the following separate score.

$$s_{c,t} = q_{0.99}(P_{c,t}) - B^{\text{base}}_{c,t}. \tag{2}$$

The emitted protection and the existing correction are excluded from this reference. If scoring used the full emitted budget, increasing $\gamma_{c,t}$ would reduce the recorded error for the same realized tail. DeCal instead lets protection cover current risk while the calibration buffer learns the base-estimator residual. Selective transfer may initialize or recover that base estimator, but it never changes the score reference. The contribution is a workload-control decomposition with a stable reference object rather than a new detector or predictor.

The invariant yields a concrete online path. At the start of each window, DeCal forms $B^{\text{base}}_{c,t}$ from completed local history or a bounded transfer prior, updates protection from completed-window drift evidence, and emits $B_{c,t}$. After the window closes, the realized distribution updates the base estimator, drift state, and calibration-score buffer. The stored score always follows Eq. (2). The online path runs once per window, so its output is an upstream capacity signal for admission control, quota, or scaling policies.

Overview of results. We evaluate DeCal with completed-history replay on ServeGen and BurstGPT. Each method receives only completed histograms and emits one tail-length proxy budget before the target window is revealed. We then score the budget against the realized P99 tail. The evaluation combines the main capacity-pressure comparison with a selected-entity analysis and two supporting checks. Together, these experiments examine aggregate performance, entity-level behavior, cold-start transfer, and the connection between token budgets and KV-cache pressure.

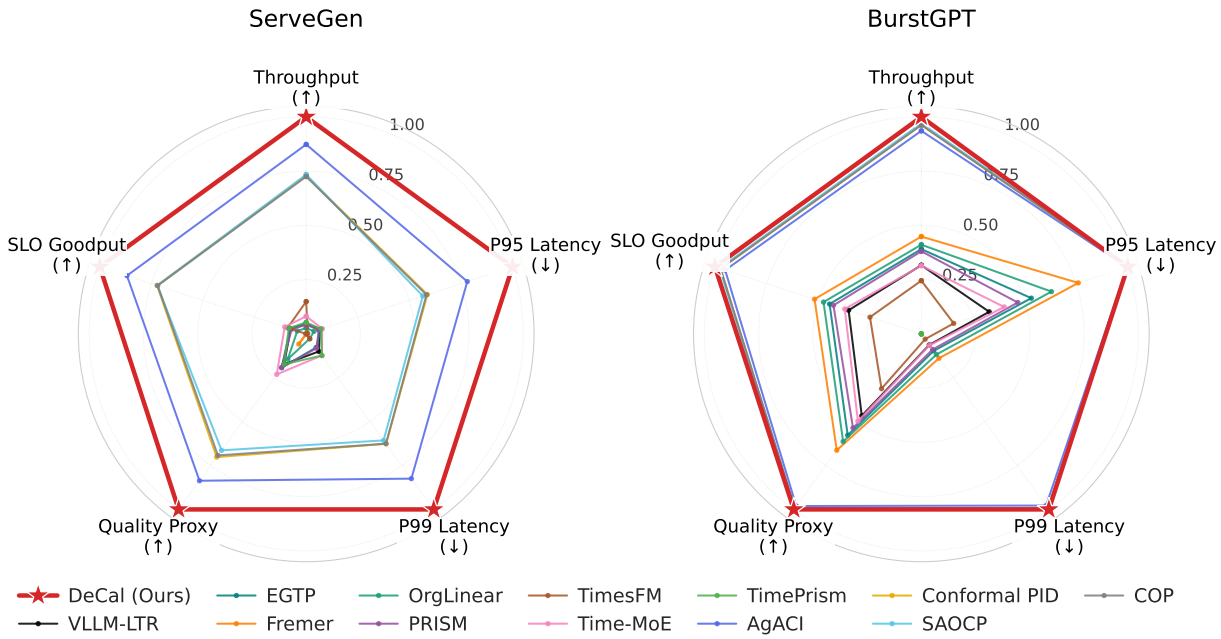


Figure 1. Standardized replay-derived capacity-pressure indicators. Latency axes are direction-adjusted, so outward is better. All indicators derive from the same budget sequences and are not independent engine measurements.

Figure 1 presents the main experimental results using five standardized replay-derived indicators. They cover throughput, P95 latency, P99 latency, Quality Proxy, and SLO Goodput. DeCal forms the outer envelope on both workloads, while prediction and forecasting baselines remain closer to the center. On ServeGen, AgACI is the strongest baseline across all five indicators. Relative to AgACI, DeCal raises throughput from 22,405.28 to 22,663.65 token/s and SLO Goodput from 21,348.47 to 21,748.68 token/s. It also reduces P95 latency by 8.63% and P99 latency from 4,374.10 to 4,329.73 ms. The larger P95 reduction shows that the gain is not confined to the extreme tail. On BurstGPT, different online-calibration baselines provide the second-best values on different axes, yet DeCal ranks first on all five. Its P99 latency falls from the best baseline value of 2,569.08 to 2,557.35 ms, while throughput and SLO Goodput rise to 4,746.49 and 4,727.55 token/s. The ServeGen gains are larger, whereas the BurstGPT gains are smaller but consistent across every axis. This pattern indicates a better mapped capacity-pressure trade-off rather than isolated optimization of one metric.

Figure 2 separates selected-entity behavior for ServeGen clients into two levels. The window-level panel reports the gap between emitted budget and realized P99 demand, where negative values indicate under-budgeting. DeCal stays at or above zero, while AgACI and SAOCP fall below demand for most displayed entities. The request-level panel reports request mass above the budget. DeCal keeps this mass near zero, whereas the baselines leave substantial demand uncovered for several entities and exceed 80% in the most exposed case. Stronger window-level coverage therefore translates

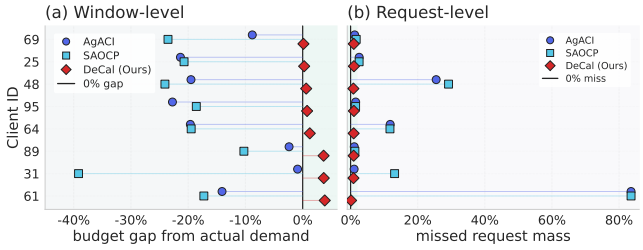


Figure 2. Selected-entity behavior for ServeGen clients at two levels. Window-level budget gaps below zero indicate under-budgeting. Request-level missed mass reports demand above the emitted budget.

into less request-level demand outside the budget. The figure diagnoses the full controller rather than one component. Overall, the results support DeCal at aggregate and entity levels. In ServeGen cold start, aggregate absolute percentage error falls from 3.23 without transfer to 0.24 with one donor and 0.22 with three. A 2x shared pool in the KV-cache pressure bridge leaves 17.7% overflow at 53.6% mean utilization. These checks support selective transfer and budget interpretability, although they do not establish end-to-end scheduler performance. Throughout replay, current-window P99 values and drift labels appear only after budget emission. The evidence therefore supports DeCal's central principle of protecting current budgets while learning only unprotected base-budget residuals.

Artifact availability. Our code is available on https://anonymous.4open.science/r/DeCal-B2B2.

References

[1] Anastasios Angelopoulos, Emmanuel Candes, and Ryan J Tibshirani. 2023. Conformal PID Control for Time Series Prediction. In *Proceedings of the 37th Advances in Neural Information Processing Systems*, Vol. 36. 23047–23074.

[2] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. In *Proceedings of the 38th Advances in Neural Information Processing Systems*, Vol. 37. 59006–59029.

[3] Isaac Gibbs and Emmanuel Candes. 2021. Adaptive Conformal Inference Under Distribution Shift. In *Proceedings of the 35th Advances in Neural Information Processing Systems*, Vol. 34. 1660–1672.

[4] Isaac Gibbs and Emmanuel J. Candès. 2024. Conformal Inference for Online Prediction with Arbitrary Distribution Shifts. *Journal of Machine Learning Research* 25, 162 (2024), 1–36.

[5] Dongjian Hu, Junxi Wu, Shu-Tao Xia, and Changliang Zou. 2026. Distribution-informed Online Conformal Prediction. In *Proceedings of the 14th International Conference on Learning Representations*.

[6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[7] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2025. Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving. *ACM Transactions on Storage* (2025).

[8] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: Dynamic Scheduling for Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*. 173–191.

[9] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchu Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*. 5831–5841.

[10] Yuxing Xiang, Xue Li, Kun Qian, Yan Zhang, Wenyuan Yu, Ennan Zhai, Xin Jin, and Jingren Zhou. 2026. ServeGen: Workload Characterization and Generation of Large Language Model Serving in Production. In *Proceedings of the 23rd USENIX Symposium on Networked Systems Design and Implementation*. 1845–1859.

[11] Huanyi Xie, Yubin Chen, Liangyu Wang, Lijie Hu, and Di Wang. 2026. Predicting LLM Output Length via Entropy-Guided Representations. In *Proceedings of the 14th International Conference on Learning Representations*, Vol. 2026.

[12] Hengyu Ye, Jiadong Chen, Fuxin Jiang, Xiao He, Tieying Zhang, Jianjun Chen, and Xiaofeng Gao. 2025. Fremer: Lightweight and Effective Frequency Transformer for Workload Forecasting in Cloud Services. *Proceedings of the VLDB Endowment* 18, 11 (2025), 3812–3825.

[13] Margaux Zaffran, Olivier Feron, Yannig Goude, Julie Josse, and Aymeric Dieuleveut. 2022. Adaptive Conformal Predictions for Time Series. In *Proceedings of the 39th International Conference on Machine Learning*, Vol. 162. 25834–25866.

[14] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation*. 193–210.